Remove Islands

AlgoExpert · Medium · Graphs

The problem

- Set to 0 every group of 1s not connected (4-directionally) to the matrix border.
- Flood-fill from border 1s marking them, then clear the rest.

Intuition

- Ones connected to the border survive; everything else is an island to remove.

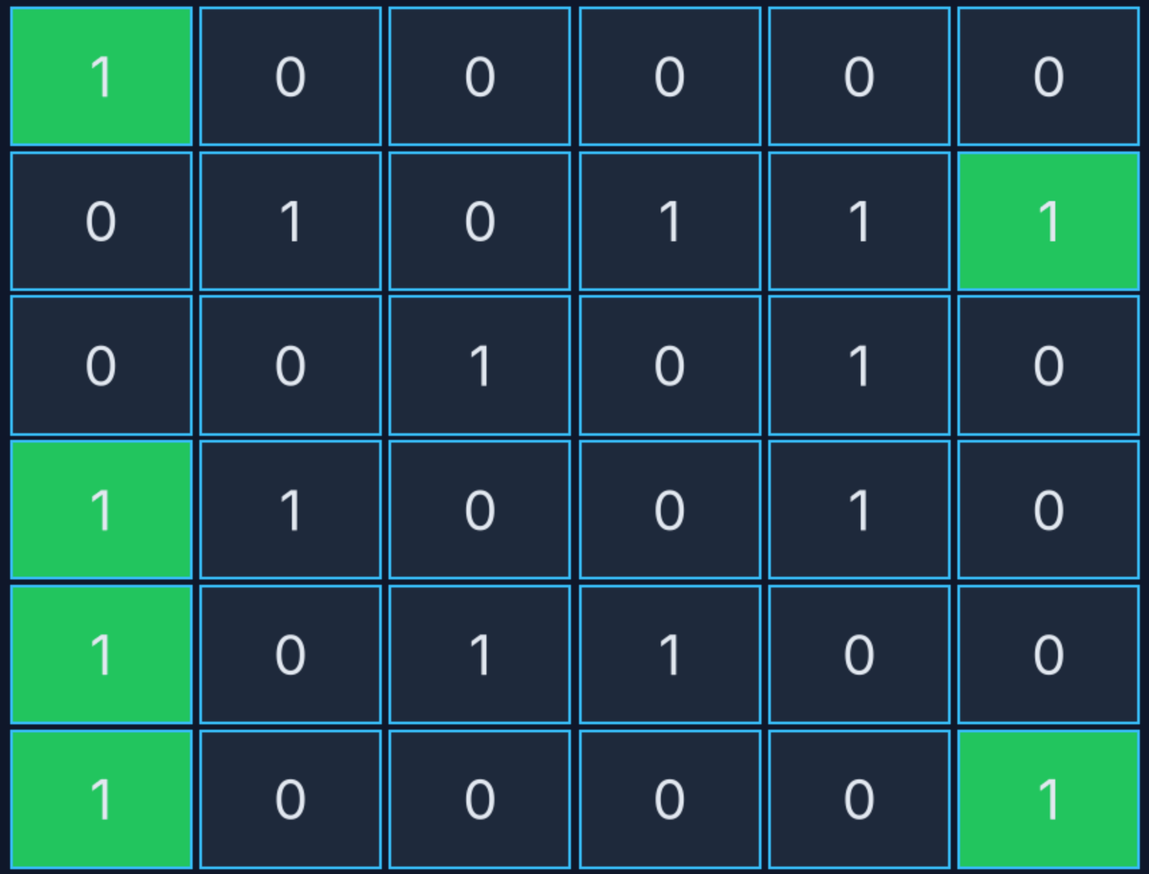
Brute force vs optimal

- Brute force: DFS from each interior 1 to see if it reaches the border → $O((w \cdot h)^2)$
- Flip the question: flood-fill from the border → $O(w \cdot h)$
- Mark safe cells in place (2), then rewrite

Approach

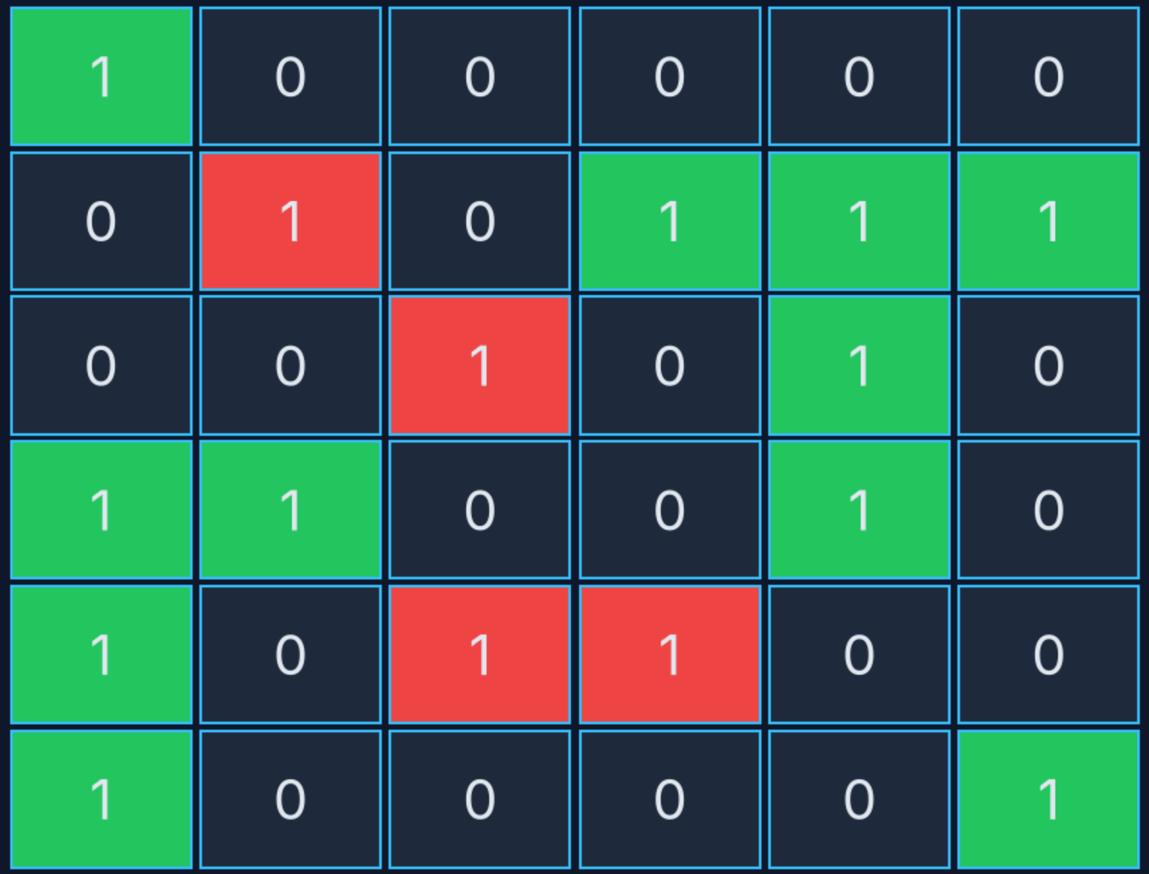
- 1. Flood-fill from every border one, marking them safe
- 2. Turn the remaining ones into zeros
- 3. Restore the safe marks

Worked example



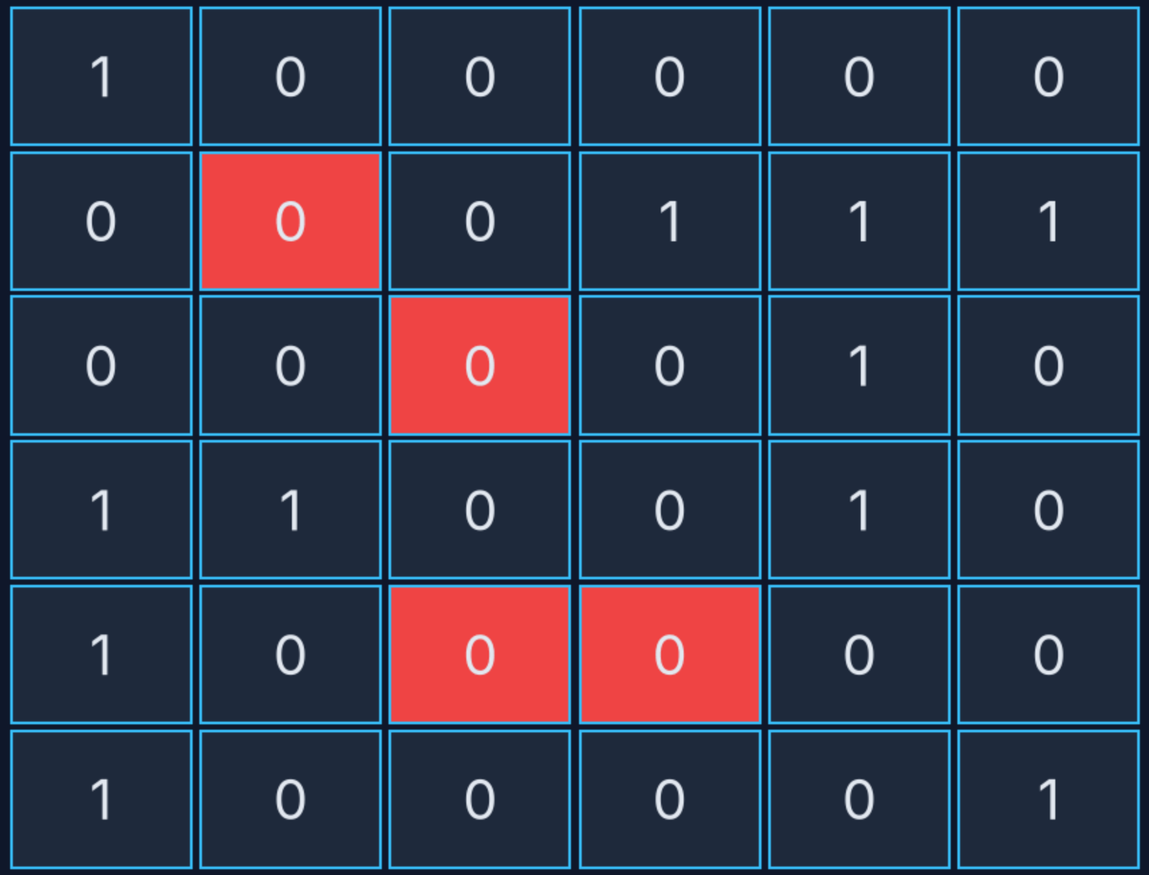
Border 1s are safe (not islands)

Worked example



Flood fill done: red cells are islands

Worked example



Islands removed

Java solution

```java
public class RemoveIslands {
    public static int[][] removeIslands(int[][] m) {
        int R = m.length, C = m[0].length;
        ArrayDeque<int[]> st = new ArrayDeque<>();
        for (int r = 0; r < R; r++) for (int c = 0; c < C; c++)
            if ((r == 0 || c == 0 || r == R - 1 || c == C - 1) && m[r][c] == 1) { m[r][c] = 2; st.push(new int[]{r, c}); }
        int[][] dirs = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
        while (!st.isEmpty()) {
            int[] p = st.pop();
            for (int[] d : dirs) {
                int r = p[0] + d[0], c = p[1] + d[1];
                if (r >= 0 && c >= 0 && r < R && c < C && m[r][c] == 1) { m[r][c] = 2; st.push(new int[]{r, c}); }
            }
        }
        for (int[] row : m) for (int c = 0; c < C; c++) row[c] = row[c] == 2 ? 1 : 0;
        return m;
    }
}
```

Complexity

- Time: O(w*h)
- Space: O(w*h)

Edge cases & pitfalls

- Diagonals don't connect (4-directional)
- Recursive DFS can overflow on large grids → use a stack
- Restore marks at the end

Interview tips & follow-ups

- Same idea: Surrounded Regions
- Follow-up: count islands (River Sizes)

Recap

- Ones connected to the border survive; everything else is an island to remove.
- Complexity: O(w*h)